

Dokumentacja bazy danych SQLite – baza zamówień

Projekt: baza zamówień

Silnik bazy danych: SQLite

Środowisko aplikacji: Node.js

Wersja dokumentu: 0.8

Data: 2026-08-29

1. Założenia ogólne

Baza danych będzie przechowywać główną tabelę zamówień projektu.

Zakładana liczba rekordów jest niewielka – około 500 rekordów rocznie – dlatego SQLite jest odpowiednim rozwiązaniem dla tego projektu.

Przyjęte zasady:

- nazwy pól w bazie są w języku angielskim i używają konwencji `snake_case`,
- pola tekstowe będą miały ograniczenia długości egzekwowane przez `CHECK`,
- pola logiczne będą przechowywane w SQLite jako `INTEGER` z wartościami `0` / `1`,
- daty biznesowe będą przechowywane jako tekst w formacie ISO 8601, np. `2026-08-24`,
- daty techniczne będą przechowywane jako tekst ISO 8601 z czasem, preferencyjnie w UTC,
- ceny i wartości finansowe będą przechowywane jako `INTEGER` po przemnożeniu przez `10000`,
- wszystkie ceny i wartości finansowe obsługują dokładność do 4 miejsc po przecinku,
- ilość w kilogramach jest liczbą całkowitą,
- brak jeszcze znanej wartości będzie zapisywany jako `NULL`, a nie jako `0`,
- wartości wybierane z tabel pomocniczych będą kopiowane do rekordu zamówienia, aby zachować poprawne dane historyczne.

2. Struktura głównej tabeli zamówień

Docelowa nazwa tabeli: `purchase_orders`

Tabela `purchase_orders` przechowuje pełny historyczny stan danych zamówienia. Dane wybierane z tabel pomocniczych (`products`, `suppliers`, `currencies`) są kopiowane do rekordu zamówienia jako wartości tekstowe.

Dzięki temu późniejsza zmiana nazwy towaru, dostawcy lub danych słownikowych nie zmienia danych widocznych we wcześniej utworzonych zamówieniach.

Lp.	Nazwa pola w bazie	Typ SQLite	Wymagane	Uwagi
1	<code>id</code>	INTEGER	Tak	Klucz główny z autonumeracją. W SQLite: <code>INTEGER PRIMARY KEY</code> .
2	<code>order_number</code>	TEXT	Nie	Numer zamówienia z lokalnego systemu CRM. Jeżeli podany, musi mieć dokładnie 10 znaków.
3	<code>product_name</code>	TEXT	Tak	Nazwa towaru skopiowana z tabeli <code>products</code> . Maksymalnie 100 znaków.

4	supplier_name	TEXT	Tak	Nazwa dostawcy skopiowana z tabeli suppliers. Maksymalnie 100 znaków.
5	quantity_kg	INTEGER	Tak	Ilość w kilogramach. Tylko liczby całkowite, wartość większa od 0.
6	port_price_per_kg	INTEGER	Tak	Cena portowa za 1 kg. W bazie jako wartość × 10000.
7	delivered_price_per_kg	INTEGER	Nie	Cena za 1 kg po uwzględnieniu dostawy. Uzupełniana później; początkowo NULL. Wartość × 10000.
8	order_value	INTEGER	Tak	Wartość zamówienia. Wyliczana automatycznie jako quantity_kg * port_price_per_kg, gdy oba składniki są dostępne. Pole tylko do odczytu w formularzu. Przechowywana × 10000.
8a	delivered_order_value	INTEGER	Nie	Wartość zamówienia po dostawie. Wyliczana automatycznie jako quantity_kg * delivered_price_per_kg, gdy delivered_price_per_kg jest znane; w przeciwnym razie NULL. Nie jest wymagana przy tworzeniu zamówienia. Pole tylko do odczytu w formularzu. Przechowywana × 10000.
9	currency_code	TEXT	Tak	Trzyliterowy kod waluty skopiowany z tabeli currencies, np. EUR, USD, PLN.
10	container_number	TEXT	Nie	Numer kontenera. Maksymalnie 50 znaków.
11	eta_port_date	TEXT	Nie	Planowana data dotarcia do portu. Format YYYY-MM-DD.
12	eta_destination_date	TEXT	Nie	Planowana data dotarcia do lokalnego miejsca docelowego. Format YYYY-MM-DD.
13	has_eur1_certificate	INTEGER	Nie	Informacja, czy dostępny jest dokument EUR.1. Wartość 0 lub 1.
14	batch_number	TEXT	Nie	Numer partii. Jeżeli podany, musi mieć dokładnie 13 znaków.
15	sent_for_testing_date	TEXT	Nie	Data wysłania partii do badań. Format YYYY-MM-DD.
16	test_results	TEXT	Nie	Wynik / informacja o wynikach badań. Maksymalnie 50 znaków.
17	is_blocked	INTEGER	Nie	Informacja o blokadzie rekordu / partii. Wartość 0 lub 1.
18	taken_for_production	INTEGER	Nie	Informacja, czy towar / partia została pobrana do produkcji. Wartość 0 lub 1.

19	payment_due_date	TEXT	Nie	Termin płatności. Format YYYY-MM-DD.
20	invoice_number	TEXT	Nie	Numer faktury. Maksymalnie 30 znaków.
21	payment_date	TEXT	Nie	Data dokonania płatności. Format YYYY-MM-DD.
22	delivery_date	TEXT	Nie	Faktyczna data dostawy. Format YYYY-MM-DD.
23	is_important	INTEGER	Nie	Ręczna flaga „ważne / wymaga uwagi” ustawiana przez użytkownika. Wartość 0 lub 1.
24	notes	TEXT	Nie	Uwagi dodatkowe. Maksymalnie 512 znaków.
25	created_at	TEXT	Tak	Data i czas utworzenia rekordu, ISO 8601. Zapisywana w UTC.
26	created_by	TEXT	Tak	Login lub inny stabilny identyfikator użytkownika, który utworzył rekord.
27	updated_at	TEXT	Nie	Data i czas ostatniej modyfikacji rekordu, ISO 8601.
28	updated_by	TEXT	Nie	Login lub inny stabilny identyfikator użytkownika, który jako ostatni zmodyfikował rekord.

3. Opis pól

Nazwa pola w bazie	Opis
id	Unikalny, automatycznie nadawany identyfikator rekordu zamówienia.
order_number	Numer zamówienia pochodzący z lokalnego systemu CRM klienta.
product_name	Nazwa towaru skopiowana ze słownika towarów w chwili tworzenia zamówienia.
supplier_name	Nazwa dostawcy skopiowana ze słownika dostawców w chwili tworzenia zamówienia.
quantity_kg	Zamawiana ilość towaru wyrażona w pełnych kilogramach.
port_price_per_kg	Cena jednostkowa za kilogram na etapie portowym.
delivered_price_per_kg	Cena jednostkowa za kilogram po poznaniu pełnych kosztów dostarczenia.
order_value	Całkowita wartość zamówienia, wyliczana automatycznie z ilości i ceny portowej.
delivered_order_value	Całkowita wartość zamówienia po dostawie, wyliczana automatycznie z ilości i ceny po dostawie.
currency_code	Trzyliterowy kod waluty skopiowany ze słownika walut w chwili tworzenia zamówienia.
container_number	Numer kontenera przypisanego do transportu zamówienia.
eta_port_date	Przewidywana data dotarcia transportu do portu.

eta_destination_date	Przewidywana data dotarcia transportu do lokalnego miejsca docelowego.
has_eur1_certificate	Informacja, czy dla zamówienia / dostawy dostępny jest dokument EUR.1.
batch_number	Numer partii towaru.
sent_for_testing_date	Data przekazania towaru lub próbki do badań.
test_results	Krótką informacją o wyniku badań.
is_blocked	Informacja, czy zamówienie / partia jest aktualnie zablokowana.
taken_for_production	Informacja, czy towar został już pobrany do produkcji.
payment_due_date	Termin, do którego powinna zostać zrealizowana płatność.
invoice_number	Numer faktury związanej z zamówieniem.
payment_date	Faktyczna data wykonania płatności.
delivery_date	Faktyczna data dostarczenia zamówienia.
is_important	Ręczna flaga „ważne / wymaga uwagi” ustawiana przez użytkownika w formularzu wpisu (FR-012).
notes	Dodatkowe uwagi dotyczące zamówienia.
created_at	Znacznik czasu utworzenia rekordu.
created_by	Użytkownik, który utworzył rekord.
updated_at	Znacznik czasu ostatniej zmiany rekordu.
updated_by	Użytkownik, który jako ostatni zmodyfikował rekord.

4. Ustalenia dotyczące wartości finansowych

Pola:

- port_price_per_kg ,
- delivered_price_per_kg ,
- order_value ,
- delivered_order_value

będą obsługiwały maksymalnie 4 miejsca po przecinku.

W SQLite wartości będą przechowywane jako liczby całkowite po przemnożeniu przez 10000 .

Przykłady:

Wartość biznesowa	Wartość w SQLite
2,3765	23765
12,5000	125000
0,0123	123

23765,0000	237650000
------------	-----------

Separatory dziesiętne używane w interfejsie użytkownika nie są częścią formatu zapisu w bazie. Konwersja pomiędzy polskim przecinkiem dziesiętnym a wartością wewnętrzną będzie realizowana przez aplikację.

5. Ustalenia dotyczące wartości opcjonalnych

Jeżeli wartość nie jest jeszcze znana, baza przechowuje `NULL`.

Przykład:

```
delivered_price_per_kg = NULL
```

oznacza:

koszt dostarczenia nie został jeszcze ustalony.

Nie należy używać wartości `0` do oznaczania braku danych, ponieważ `0` jest prawidłową wartością liczbową i ma inne znaczenie biznesowe.

6. Tabele pomocnicze

Tabele pomocnicze pełnią rolę słowników do wyboru wartości w aplikacji.

Ważna zasada projektowa:

Rekord zamówienia nie przechowuje identyfikatorów rekordów z tabel pomocniczych. Wybrana wartość jest kopiowana do tabeli `purchase_orders`.

Dzięki temu dane historyczne są niezależne od późniejszych zmian w słownikach.

6.1. Tabela towarów – `products`

Pole	Typ SQLite	Wymagane	Uwagi
<code>id</code>	INTEGER	Tak	Klucz główny z autonumeracją.
<code>name</code>	TEXT	Tak	Nazwa towaru, maksymalnie 100 znaków. Wartość unikalna.

6.2. Tabela dostawców – `suppliers`

Pole	Typ SQLite	Wymagane	Uwagi
<code>id</code>	INTEGER	Tak	Klucz główny z autonumeracją.
<code>name</code>	TEXT	Tak	Nazwa dostawcy, maksymalnie 100 znaków. Wartość unikalna.

6.3. Tabela walut – `currencies`

Pole	Typ SQLite	Wymagane	Uwagi
<code>id</code>	INTEGER	Tak	Klucz główny z autonumeracją.

code	TEXT	Tak	Trzyliterowy kod waluty ISO 4217, np. EUR, USD, PLN. Wartość unikalna.
------	------	-----	--

6.4. Tabela historii zmian – `purchase_order_history` (FR-010)

Rejestruje kto i kiedy edytował dany wpis oraz które pola się zmieniły. Wpis w tej tabeli powstaje wyłącznie przy edycji istniejącego zamówienia (nie przy jego utworzeniu) i tylko wtedy, gdy przynajmniej jedno pole faktycznie się zmieniło.

Pole	Typ SQLite	Wymagane	Uwagi
id	INTEGER	Tak	Klucz główny z autonumeracją.
order_id	INTEGER	Tak	Klucz obcy do <code>purchase_orders.id</code> , ON DELETE CASCADE.
edited_by	TEXT	Tak	Login użytkownika, który zapisał edycję.
edited_at	TEXT	Tak	Znacznik czasu UTC, ustawiany automatycznie przez SQLite.
changes	TEXT	Tak	JSON — tablica obiektów { label, oldValue, newValue }, po jednym na każde zmienione pole. Wartości są już sformatowane do wyświetlenia (np. kwoty przez <code>formatMoney</code> , flagi jako „Tak”/„Nie”).

W przeciwieństwie do pozostałych tabel, `changes` przechowuje dane zdenormalizowane (gotowe do wyświetlenia), a nie surowe wartości kolumn — historia jest wyłącznie do odczytu i nie musi być zgodna z ewentualną przyszłą zmianą formatu w aplikacji.

7. Skrypt inicjalizujący bazę SQLite

SQLite nie posiada osobnej instrukcji `CREATE DATABASE`. Plik bazy jest tworzony podczas otwarcia wskazanego pliku przez SQLite / Node.js, a poniższy skrypt `init.sql` tworzy strukturę tabel, ograniczenia oraz indeksy.

```
BEGIN TRANSACTION;

CREATE TABLE IF NOT EXISTS products (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL
    CHECK (length(trim(name)) BETWEEN 1 AND 100),
  UNIQUE (name)
);

CREATE TABLE IF NOT EXISTS suppliers (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL
    CHECK (length(trim(name)) BETWEEN 1 AND 100),
  UNIQUE (name)
);

CREATE TABLE IF NOT EXISTS currencies (
  id INTEGER PRIMARY KEY,
  code TEXT NOT NULL
    CHECK (
```

```

        length(code) = 3
        AND code = upper(code)
    ),
    UNIQUE (code)
);

CREATE TABLE IF NOT EXISTS purchase_orders (
    id INTEGER PRIMARY KEY,

    order_number TEXT
        CHECK (
            order_number IS NULL
            OR length(order_number) = 10
        ),

    product_name TEXT NOT NULL
        CHECK (length(trim(product_name)) BETWEEN 1 AND 100),

    supplier_name TEXT NOT NULL
        CHECK (length(trim(supplier_name)) BETWEEN 1 AND 100),

    quantity_kg INTEGER NOT NULL
        CHECK (quantity_kg > 0),

    port_price_per_kg INTEGER NOT NULL
        CHECK (port_price_per_kg >= 0),

    delivered_price_per_kg INTEGER
        CHECK (
            delivered_price_per_kg IS NULL
            OR delivered_price_per_kg >= 0
        ),

    order_value INTEGER NOT NULL
        CHECK (order_value >= 0),

    delivered_order_value INTEGER
        CHECK (
            delivered_order_value IS NULL
            OR delivered_order_value >= 0
        ),

    currency_code TEXT NOT NULL
        CHECK (
            length(currency_code) = 3
            AND currency_code = upper(currency_code)
        ),

    container_number TEXT
        CHECK (
            container_number IS NULL
            OR length(container_number) <= 50

```

```

    ),

    eta_port_date TEXT,
    eta_destination_date TEXT,

    has_eur1_certificate INTEGER
        CHECK (
            has_eur1_certificate IS NULL
            OR has_eur1_certificate IN (0, 1)
        ),

    batch_number TEXT
        CHECK (
            batch_number IS NULL
            OR length(batch_number) = 13
        ),

    sent_for_testing_date TEXT,

    test_results TEXT
        CHECK (
            test_results IS NULL
            OR length(test_results) <= 50
        ),

    is_blocked INTEGER
        CHECK (
            is_blocked IS NULL
            OR is_blocked IN (0, 1)
        ),

    taken_for_production INTEGER
        CHECK (
            taken_for_production IS NULL
            OR taken_for_production IN (0, 1)
        ),

    payment_due_date TEXT,

    invoice_number TEXT
        CHECK (
            invoice_number IS NULL
            OR length(invoice_number) <= 30
        ),

    payment_date TEXT,
    delivery_date TEXT,

    is_important INTEGER
        CHECK (
            is_important IS NULL
            OR is_important IN (0, 1)

```

```

    ),

    notes TEXT
        CHECK (
            notes IS NULL
            OR length(notes) <= 512
        ),

    created_at TEXT NOT NULL
        DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),

    created_by TEXT NOT NULL
        CHECK (length(trim(created_by)) > 0),

    updated_at TEXT,
    updated_by TEXT
);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_is_important
    ON purchase_orders(is_important);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_order_number
    ON purchase_orders(order_number);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_product_name
    ON purchase_orders(product_name);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_supplier_name
    ON purchase_orders(supplier_name);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_currency_code
    ON purchase_orders(currency_code);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_batch_number
    ON purchase_orders(batch_number);

CREATE INDEX IF NOT EXISTS idx_purchase_orders_container_number
    ON purchase_orders(container_number);

CREATE TABLE IF NOT EXISTS purchase_order_history (
    id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL
        REFERENCES purchase_orders(id) ON DELETE CASCADE,
    edited_by TEXT NOT NULL,
    edited_at TEXT NOT NULL
        DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
    changes TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_purchase_order_history_order_id
    ON purchase_order_history(order_id);

```

```
COMMIT;
```

Uwagi do skryptu

1. `purchase_orders.id` ma autonumerację dzięki `INTEGER PRIMARY KEY`.
2. Nie zastosowano słowa `AUTOINCREMENT`, ponieważ w SQLite `INTEGER PRIMARY KEY` automatycznie nadaje kolejne identyfikatory i jest wystarczające dla tego projektu.
3. Tabele `products`, `suppliers` i `currencies` są słownikami pomocniczymi.
4. Tabela `purchase_orders` nie posiada kluczy obcych do tabel słownikowych.
5. Przy tworzeniu zamówienia aplikacja kopiuje wybrane wartości słownikowe do `product_name`, `supplier_name` i `currency_code`.
6. Dzięki kopiowaniu danych późniejsza zmiana tabel pomocniczych nie wpływa na wcześniej zapisane zamówienia.
7. `created_at` jest ustawiane automatycznie przez SQLite w UTC.
8. `created_by`, `updated_at` i `updated_by` powinny być ustawiane przez aplikację Node.js.
9. `updated_at` pozostaje `NULL` do pierwszej modyfikacji rekordu.
10. `delivered_price_per_kg` pozostaje `NULL`, dopóki dokładny koszt dostarczenia nie jest znany.
11. Pola finansowe są przechowywane jako liczby całkowite pomnożone przez `10000`.
12. `order_value` jest wyliczane automatycznie przez aplikację jako `quantity_kg * port_price_per_kg`; pole jest tylko do odczytu w formularzu.
13. `delivered_order_value` jest wyliczane automatycznie przez aplikację jako `quantity_kg * delivered_price_per_kg`, gdy `delivered_price_per_kg` jest znane, w przeciwnym razie pozostaje `NULL`; pole jest tylko do odczytu w formularzu i nie jest wymagane przy tworzeniu zamówienia.
14. Mimo braku klucza obcego z `purchase_orders.currency_code` do `currencies` (patrz punkt 4), aplikacja przy zapisie wpisuje waliduje, że podany kod waluty istnieje w tabeli `currencies` — odrzuca zapis, jeśli waluta nie jest w słowniku, nawet jeśli sam format (3 wielkie litery) jest poprawny.
15. `order_number` i `batch_number` nie mają w bazie ograniczenia `UNIQUE`, ale aplikacja przy zapisie sprawdza, czy podana wartość nie jest już użyta w innym wpisie (z wyłączeniem samego edytowanego rekordu), i odrzuca zapis w razie duplikatu.

8. Reguła „wymaga uwagi” (FR-012 i logika biznesowa)

Wpis jest oznaczany jako „wymagający uwagi” w widoku tabelarycznym i w raporcie, gdy spełniony jest co najmniej jeden z warunków:

1. `is_important = 1` — użytkownik ręcznie oznaczył wpis jako ważny.
2. `eta_destination_date` jest ustawione, `delivery_date` jest `NULL`, a do `eta_destination_date` zostały co najwyżej 3 dni (lub termin już minął) — dostawa się zbliża (lub jest opóźniona), a nie została jeszcze potwierdzona.
3. `delivery_date` jest ustawione, a `test_results` jest `NULL` — dostawa się zakończyła, ale nie wprowadzono jeszcze wyników badań.

Flaga nie jest przechowywana jako osobna kolumna wynikowa — jest wyliczana w zapytaniu SQL w momencie odczytu (`date('now', '+3 days')`), ponieważ zależy od bieżącej daty, a przechowywanie wyniku groziłoby jego nieaktualnością.

9. Wersjonowane migracje schematu

`CREATE TABLE/INDEX IF NOT EXISTS` w skrypcie inicjalizującym pozostaje wystarczające dla nowych tabel i indeksów — jest bezpieczne zarówno na świeżej, jak i na już istniejącej bazie. Nie wystarcza jednak dla zmian w już istniejącej tabeli (nowa kolumna, zmiana ograniczenia), bo SQLite nie ma odpowiednika `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`.

Takie zmiany są od wersji 0.8 aplikowane przez prosty mechanizm wersjonowanych migracji (`src/lib/migrations.ts`):

1. Tabela `schema_migrations` (`version`, `name`, `applied_at`) przechowuje log zastosowanych migracji.
2. Każda migracja to numerowany krok z funkcją `up(db)`, idempotentną wewnątrznie (sprawdza `PRAGMA table_info` przed `ALTER TABLE`, na wypadek gdyby kolumna już istniała ze świeżego schematu).
3. Przy starcie aplikacji uruchamiane są tylko migracje o numerze wyższym niż najwyższy już zapisany w `schema_migrations`, każda w osobnej transakcji.
4. Na świeżej bazie migracje stają się no-op (kolumny już są w bazowym `CREATE TABLE`), ale i tak zostają odnotowane — świeża i zaktualizowana-w-miejsu baza kończą z identyczną historią `schema_migrations`.

Dotychczasowe dwa ręczne dopasowania kolumn (`is_important` z wersji 0.4, `delivered_order_value` z modelu finansowego) zostały przekształcone w migracje nr 1 i 2. Każda kolejna zmiana kolumny/ograniczenia w istniejącej tabeli powinna trafić jako nowy, kolejny numer w tej samej liście — nie jako kolejny ręczny `PRAGMA table_info` + `ALTER TABLE` w `db.ts`.

Zweryfikowano end-to-end (świeża baza, symulowana baza sprzed migracji z realnymi danymi, podwójne uruchomienie) — dane nie są tracone, kolumny/indeksy odtwarzane poprawnie, mechanizm idempotentny.

10. Tematy do dalszego ustalenia

W kolejnych wersjach dokumentacji należy doprecyzować między innymi:

- czy `container_number` ma podlegać walidacji zgodnej ze standardowym formatem numerów kontenerów.

Rozstrzygnięte od poprzedniej wersji dokumentu:

- `order_number` i `batch_number` — unikalne w praktyce: aplikacja odrzuca zapis, jeśli podana wartość jest już użyta w innym wpisie (patrz punkt 15 w Uwagach do skryptu). Brak klucza unikalnego w samej bazie — celowo, żeby uniknąć błędu przy starcie na już istniejących danych z ewentualnymi duplikatami sprzed wprowadzenia tej reguły.
- wartości początkowe tabeli `currencies`, sposób zarządzania listą towarów/dostawców, zasady `updated_at` / `updated_by`, tabela użytkowników, nazwa/lokalizacja pliku bazy — zaimplementowane (zarządzanie słownikami i użytkownikami w panelu administratora, `updated_at` / `updated_by` ustawiane przy edycji, baza w `data/mini-dm.db` konfigurowalna przez `DATABASE_PATH`).
- strategia migracji schematu bazy przy kolejnych wersjach aplikacji — patrz sekcja 9 powyżej.

11. Historia zmian

Wersja	Data	Opis
0.1	2026-08-24	Utworzenie dokumentu bazowego i zebranie pierwszych ustaleń dotyczących struktury tabeli zamówień.
0.2	2026-08-24	Dodanie tabel pomocniczych <code>products</code> , <code>suppliers</code> , <code>currencies</code> oraz pierwszej wersji skryptu inicjalizującego SQLite.

0.3	2026-08-24	Zmiana modelu danych historycznych: wartości z tabel pomocniczych są kopiowane do <code>purchase_orders</code> ; usunięto <code>product_id</code> , <code>supplier_id</code> i <code>currency_id</code> z tabeli zamówień. Potwierdzono autonumerowane pole <code>id</code> w tabeli głównej.
0.4	2026-08-25	Dodanie kolumny <code>is_important</code> (ręczna flaga „ważne”) i reguły „wymaga uwagi” (FR-012), wyliczanej w zapytaniu na podstawie <code>is_important</code> , <code>eta_destination_date</code> , <code>delivery_date</code> i <code>test_results</code> .
0.5	2026-08-27	Dodanie tabeli <code>purchase_order_history</code> (kto/kiedy edytował i które pola się zmieniły) realizującej FR-010. Historia obejmuje wyłącznie edycje, nie utworzenie wpisu.
0.6	2026-08-29	Doprecyzowanie: aplikacja waliduje przy zapisie, że <code>currency_code</code> istnieje w słowniku <code>currencies</code> (mimo braku klucza obcego, patrz punkt 4/14 w Uwagach do skryptu). Rozważana walidacja "termin dostawy nie wcześniejszy niż data złożenia zamówienia" została świadomie odrzucona — <code>created_at</code> to znacznik utworzenia rekordu, nie biznesowa data złożenia zamówienia, więc taka reguła uniemożliwiłaby uzupełnianie zaległych wpisów historycznych.
0.7	2026-08-29	Rozstrzygnięcie unikalności <code>order_number</code> i <code>batch_number</code> : aplikacja odrzuca zapis przy duplikacie (bez klucza <code>UNIQUE</code> w bazie, patrz punkt 15 w Uwagach do skryptu). Uporządkowano listę "Tematy do dalszego ustalenia" — usunięto pozycje już zaimplementowane.
0.8	2026-08-29	Wprowadzenie wersjonowanego mechanizmu migracji schematu (<code>schema_migrations</code> + <code>src/lib/migrations.ts</code> , sekcja 9) zamiast ręcznych <code>PRAGMA table_info</code> + <code>ALTER TABLE</code> w <code>db.ts</code> . Dotychczasowe dwa dopasowania kolumn (<code>is_important</code> , <code>delivered_order_value</code>) przekształcone w migracje nr 1 i 2.